CHAPTER 6

The Client–Server Model & the Request Lifecycle

Part II · Building Blocks — the skeleton almost every system is built on.

LEARNING OBJECTIVES

- Explain the client–server model and the roles each side plays.
- Describe the classic **3-tier architecture** (presentation, application, data) and why we separate tiers.
- Trace a request end-to-end through a modern multi-tier system, naming every hop.
- Explain the difference between **stateless** and **stateful** servers, and why stateless is the key to scaling.
- Identify which tier is easy to scale and which is the hard bottleneck — and why.

REAL-WORLD ANALOGY — A SERVICE DESK THAT GROWS INTO A LARGE OPERATION

Imagine a small shop with one service desk. You (the **client**) walk up and ask for something; a staff member (the **server**) fetches it from the records room (the **database**) and hands it back. As the shop gets popular, one desk isn't enough, so they add a **greeter** who directs each customer to whichever desk is free (a **load balancer**), and a **quick-answers board** for the most common questions so staff don't run to the records room every time (a **cache**). Nothing about the customer's request changed — but the operation behind the counter grew into tiers. That growth is exactly this chapter.

6.1 What Part II is about

Part I gave you the raw foundations: how one machine works, how machines talk, how the web works, and how to size a system with numbers. Part II now assembles those foundations into the **building blocks of scalable systems**. We start here with the most fundamental shape of all — clients and servers — and the path a request takes through the tiers. Every later block (load balancer, cache, gateway, database) plugs into this skeleton.

6.2 The client–server model

Almost every system you use follows one simple pattern: a **client** asks, a **server** answers. Your browser, phone app, or smart TV is the client; a machine in a data center is the server.

The client does…	The server does…
Shows the user interface; collects input; sends requests; renders responses.	Receives requests; runs the business logic; reads/writes data; sends responses.

Why is this split so universal? Because it gives **separation of concerns** and control: the server centralizes the logic and data, many clients share one authoritative system, and sensitive data stays behind the server where it can be secured — never handed to untrusted

clients. (Recall from Chapter 4 that HTTP requests are stateless — each one stands alone; that property will matter a lot in Section 6.6.)

THE ALTERNATIVE: PEER-TO-PEER

Not everything is client–server. In **peer-to-peer** (P2P) systems like BitTorrent, there is no central server — every participant is both client and server, talking directly to others. P2P shines for sharing load across users, but most business systems are client–server because they need a central, trusted authority over data. We'll focus on client–server for the rest of the book.

6.3 From one server to tiers

In Chapter 1 we saw the naive start: one machine running the app *and* the database together. It's simple but fails under growth — one point of failure, one hard limit, everything coupled. The fix is to split the system into **tiers**, each with one job, each able to scale and be secured on its own. The classic form is the **3-tier architecture**:

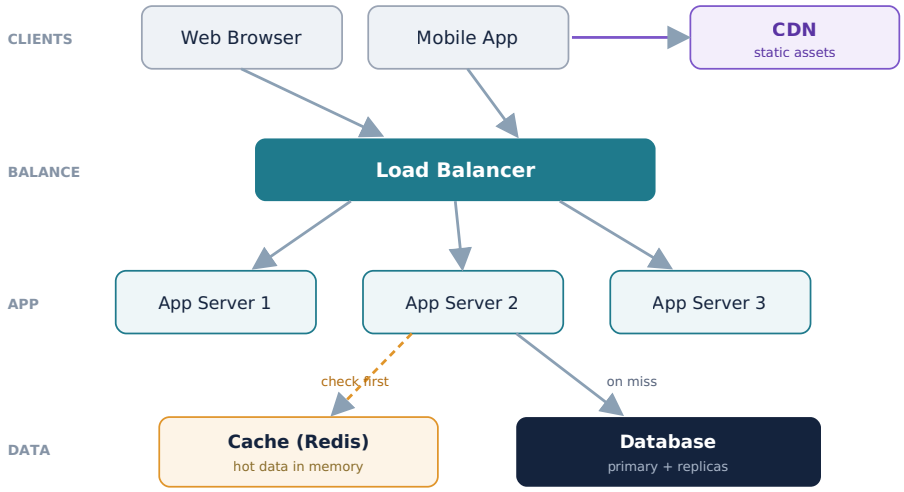
Tier	Job	Example
1. Presentation	What the user sees and interacts with.	Web/mobile frontend
2. Application (logic)	The business rules — the actual work.	Backend app servers
3. Data	Storing and retrieving data durably.	Database, object storage

KEY INSIGHT — TIERS LET EACH PART SCALE ON ITS OWN

Separating tiers means you can add app servers without touching the database, upgrade the database without redeploying the frontend, and put the data tier on a private network the internet can never reach directly. Each tier scales, deploys, and is secured independently. This separation is the foundation of every scalable design in this book.

6.4 A modern multi-tier request lifecycle

Here is what a real, scaled system looks like — the skeleton the next several chapters flesh out. Follow the arrows: this single figure contains the load balancer (Chapter 7), the cache (Chapter 8), and the CDN, all sitting on the client–server bones.



The request lifecycle, step by step

1. **Client sends a request** over HTTPS (DNS already resolved, Chapter 4). Static assets — images, CSS, JavaScript — are fetched from the **CDN** near the user, never from the app servers.
2. **The load balancer** is the public entry point. It picks a healthy app server to handle the request (Chapter 7).
3. **An app server runs the business logic.** Because app servers are *stateless* (Section 6.6), **any** of them can handle **any** request — which is exactly why we can have three, or three hundred.
4. **The app checks the cache first** (Chapter 8). A cache hit returns hot data from memory in well under a millisecond.
5. **On a cache miss, it queries the database** (the data tier), then usually writes the result back into the cache so the next request is fast.
6. **The response travels back** up through the load balancer to the client, which renders it.

This layered flow is the backbone of modern systems. The rest of Part II and Part III simply zoom into each box: the load balancer, the cache, the API gateway, horizontal scaling, and then the hard problem — scaling the data tier.

6.5 Stateless vs stateful servers

This is the most important idea in the chapter for scaling. It concerns whether a server **remembers** anything about a client between requests.

Stateless server	Stateful server
Keeps no client data between requests. Each request carries everything needed.	Remembers client data in its own memory (e.g. a login session stored locally).
Any server can handle **any** request → easy to add servers, easy failover.	A client's requests must return to the **same** server ("sticky sessions") → hard to scale, bad failover.
If one dies, another takes over instantly.	If it dies, the state it held is **lost**.

KEY INSIGHT — KEEP THE APP TIER STATELESS, PUSH STATE TO A SHARED STORE

The golden rule is to make app servers **stateless** and store any per-user state (like sessions) in a **shared** place all servers can reach — a database or a Redis session store. Then adding servers is trivial and a dying server loses nothing. This is the practical payoff of HTTP's statelessness from Chapter 4, and it's why real systems externalize sessions (Chapter 41) instead of keeping them in app-server memory.

PITFALL — STICKY SESSIONS

If you store a user's session in one server's memory, the load balancer must send that user back to that exact server every time ("session affinity" / sticky sessions). This undermines load balancing (traffic can't spread freely), breaks when that server dies, and blocks smooth scaling. Sometimes it's a pragmatic shortcut, but the durable answer is a stateless app tier with externalized state.

6.6 Where the tiers scale — and where it hurts

Tier	How hard to scale
Presentation (CDN + client)	Easy — the CDN serves static content from many edge locations.
Application (stateless app servers)	Easy — just add identical servers behind the load balancer.
Data (database)	**Hard** — you can't freely clone a database, because all copies must agree on the data.

KEY INSIGHT — WE MAKE THE APP TIER STATELESS TO ISOLATE THE HARD PROBLEM

Notice the pattern: presentation and application tiers scale almost for free, but the **data tier is the real bottleneck**, because data must stay *consistent* across copies. Keeping the app tier stateless is a deliberate move — it pushes all the hard scaling work down into one place, the data tier, which is exactly what Part III (replication, partitioning, sharding, consistency) is devoted to solving.

Common mistakes

WATCH OUT

- Storing sessions or other state in app-server memory — breaking horizontal scaling and failover.
- Exposing the database directly to clients or the public internet instead of hiding it behind the app tier.
- Coupling tiers so tightly they can't be scaled, deployed, or secured independently.
- Assuming the data tier scales like the app tier — ignoring that database copies must stay consistent.
- Routing static assets (images, CSS, JS) through app servers instead of a CDN.

- Sending traffic to dead servers because the load balancer has no health checks (Chapter 7).

Interview questions

1. Explain the client–server model and the 3-tier architecture. Why separate the tiers?
2. Trace a request from a browser all the way to the database and back in a modern architecture.
3. Why do we keep the application tier stateless? What specifically breaks if it’s stateful?
4. What are sticky sessions, and what problems do they cause?
5. Which tier is hardest to scale, and why?
6. Why shouldn’t clients talk to the database directly?
7. What is the difference between client–server and peer-to-peer architectures?

Hands-on exercises

1. Draw the 3-tier architecture for a simple blog app and label each tier’s responsibility.
2. For a “view profile” request, list every hop from client to database and back.
3. Take a system you know and classify its components as stateless or stateful.
4. Decide where session data should live so the app tier stays stateless, and justify it.
5. Sketch where a CDN fits in the lifecycle and which specific requests it should serve.

Mini project

TRACE AND TIER AN APP

Pick a simple app (say a to-do list). Draw the full modern request lifecycle: clients, CDN, load balancer, a stateless app tier, a cache, and the database. Then write out the exact numbered path taken by two requests — a `POST /todos` (create) and a `GET /todos` (list) — naming every hop and where the cache is checked. Mark which components are stateless and where any user state is stored. This cements the skeleton you’ll reuse for every design in the book.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE REFERENCE ARCHITECTURE

Using your Chapter 5 estimates, draw the **multi-tier reference architecture** for your platform: a load balancer, a stateless app tier, a cache, the database, and a CDN for media. Decide where user **session state** lives so the app tier stays stateless. Then annotate, based on your numbers, **which tier will need the most scaling** (almost certainly the data tier and the media/CDN path). Keep this as your base diagram — you’ll extend it with a real load balancer in Chapter 7, a caching strategy in Chapter 8, and database scaling in Part III.

Chapter summary

- The **client–server model** — clients ask, servers answer — is the universal shape of systems, chosen for separation of concerns, centralized data, and security.
- The **3-tier architecture** (presentation, application, data) splits a system so each tier can scale, deploy, and be secured independently.
- A modern request flows: **client → CDN (static) / load balancer → stateless app server → cache → database**, and back — the backbone the next chapters flesh out.
- **Stateless** app servers let any server handle any request, enabling easy scaling and failover; **stateful** servers force sticky sessions and lose state on failure. Externalize state to a shared store.
- Presentation and application tiers scale easily; the **data tier is the hard bottleneck** because copies must stay consistent — the subject of Part III.

COMING NEXT

Chapter 7 — Load Balancing & Reverse / Forward Proxies. We zoom into the box that made the app tier scalable. How does the load balancer choose a server? How does it know which servers are alive? What’s the difference between a reverse proxy and a forward proxy, and between Layer 4 and Layer 7 balancing? That’s next.